FINPILOT AI - AN AGENTIC AI-BASED AUTONOMOUS PERSONAL FINANCIAL PLANNING AND DECISION SUPPORT SYSTEM

A PROJECT REPORT FOR THE COURSE

AD23731 - FOUNDATIONS OF AGENTIC AI

Submitted by

Lokaa V (231501085)

Marcellus Jesudasan (231501096)

Mohammad Iftikar (231501101)

IVth YEAR - B.Tech



Department of Artificial Intelligence and Machine Learning

RAJALAKSHMI ENGINEERING COLLEGE

Thandalam,

Chennai - 602105

OCT - 2026

Abstract

FinPilot AI is a personal financial planning and decision support application that combines a deterministic financial engine with a stateful agent workflow. The system accepts financial profiles, transaction records, recurring commitments, and savings goals. It calculates spending patterns, available cash flow, emergency coverage, goal requirements, and the impact of alternative decisions. Specialized agents interpret these results and present planning recommendations through a web interface.

The implementation uses React and TypeScript for the frontend, FastAPI for backend services, SQLAlchemy for persistence, and LangGraph for task coordination. PostgreSQL is the intended deployment database; SQLite supports local verification. Groq provides the configurable language model interface, while a deterministic mock response mode supports offline execution. Numerical methods use Python Decimal values for financial calculations. The graph includes intent routing, spending analysis, decision evaluation, planning, monitoring, conditional replanning, output checks, and bounded reflection.

A local knowledge component stores four curated financial knowledge documents containing twelve chunks. Retrieval combines 384 dimensional hashed text vectors, keyword overlap, category filtering, and one fallback search. Citations and retrieval events are attached to execution state. Human review endpoints record acceptance, modification, or rejection of recommendations. Persistent decision memory enables previous choices and their assumptions to be reconsidered against current financial conditions.

Verification of repository revision 80332ca68f0f on 2 October 2026 produced 169 passing backend tests in SQLite and mock language model mode, and a successful frontend production build. Additional targeted probes nevertheless exposed salary double counting, omission of the configured emergency reserve, ineffective numerical fact checking, and Agent Lab routing discrepancies. Retrieved passages are stored and returned but are not supplied to the inspected specialist generation call. The prototype therefore runs under the tested conditions, while complete financial correctness and end to end answer grounding remain unfinished. Demonstration benchmark values are excluded from measured results.

Keywords

Agentic AI; personal financial planning; LangGraph; deterministic financial engine; decision memory; agentic retrieval; human in the loop; adaptive replanning.

Report scope

The system is evaluated through repository tests, a frontend build, and targeted API and component probes. Worked financial examples use synthetic values. Spaces are reserved for architecture figures and implementation screenshots; these figures should be added from the verified application before submission.

Content

Section Title	Page Range
1. Introduction	4
2. Problem Statement and Business Case	5
3. User and Stakeholder Analysis	5
4. Proposed Agentic AI Solution	6
5. Dataset and Knowledge Sources	7
6. Tools and Technologies Used	8
7. Agent Architecture and Calculation Methods	9-12
8. Agent Workflow, Memory, Monitoring, and Human Review	13-14
9. Agentic RAG Implementation and Quality Evaluation	15-16
10. System Implementation and Repository	17-20
11. Experimental Evaluation and Results	21-22
12. Comparison Innovation and Scalability	23
13. Conclusion and Future Work	23
References	24

Figures

Figure 1 System architecture — page 9
Figure 2 Conditional orchestration workflow — page 13
Figure 3 Financial dashboard — page 17
Figure 4 Goal and plan interface — page 18
Figure 5 Agent Lab interface — page 19

Evidence and reproducibility

Primary implementation evidence is the FinPilot AI source repository [1]. Module names and formulas are taken from source code. Verification was performed on 2 October 2026 using the revision identified in the evaluation chapter. References [2]–[7] identify the foundational technologies and retrieval literature relevant to the design.

1 Introduction

1.1 Background and motivation

Personal financial planning requires several connected decisions. A person must account for income, unavoidable expenses, debt commitments, savings reserves, and goals that compete for the same monthly surplus. Recording transactions provides useful evidence, but the user must still interpret that evidence and decide whether a purchase is affordable or a savings target is realistic. A plan can also become outdated when income falls, recurring expenses increase, or an unexpected expense reduces available savings.

FinPilot AI addresses this problem through a common workspace for financial data, numerical analysis, scenario testing, and explanatory assistance. A user can review spending, define goals, ask a question, and inspect the consequences of alternative actions. The motivation is to connect these activities so that an explanation refers to the same profile, transactions, and goals used by the calculations.

1.2 Need for Agentic AI

A financial query often requires more than generating a textual answer. A purchase question requires parameter extraction, a current financial snapshot, scenario calculation, and interpretation of effects on reserves and goals. A plan health question requires comparison with a baseline and may need a revised allocation. Agentic coordination provides explicit stages for these tasks and records which components participated in producing a response.

In this implementation, autonomy is bounded by graph transitions, rule based routing, available calculation functions, and user review. The system can select a supported processing path and propose an adaptation. It does not execute bank transfers or make financial commitments on behalf of the user. Monitoring is evaluated through application requests and state comparisons; an independently scheduled background monitoring service is not established by the reviewed code.

1.3 Project objectives

The project objectives are to consolidate financial inputs; calculate cash flow and spending indicators; evaluate savings goals and competing priorities; compare purchase and income change scenarios; coordinate specialist processing through shared state; retain useful decision history; and present recommendations with an execution trace. A further objective is to support local knowledge retrieval and human review so that the prototype can be inspected and evaluated as an agentic application.

2 Problem Statement and Business Case

2.1 Problem definition

The problem is the absence of a connected decision process between a user's financial records and an actionable plan that can be revised. A user may have sufficient account balance for a purchase but insufficient remaining liquidity to protect recurring obligations or reach a goal. Independent calculations, transaction dashboards, and generic conversational advice can leave these relationships unclear.

2.2 Real world context and expected value

FinPilot is relevant to students with limited budgets, employees managing salary cycles, and households balancing savings goals with recurring payments. Its expected value is clearer visibility of trade offs and faster scenario exploration. For example, postponing a purchase or reducing discretionary expenses can be compared against the effect on emergency coverage and target dates. These are intended benefits; the project does not include a longitudinal study establishing improved savings behavior or commercial return.

3 User and Stakeholder Analysis

3.1 Target users and stakeholders

The primary stakeholder is the individual whose financial profile and records drive the plan. The development team maintains calculation rules, API contracts, and knowledge data. Academic evaluators assess architecture, implementation, and test evidence. Knowledge source maintainers influence the quality of retrieved guidance. Each stakeholder requires a different form of transparency: understandable advice for users, traceable execution for developers, and reproducible evidence for evaluators.

3.2 User requirements

Requirement	Implementation Response
Understand Spending	Category breakdowns, recurring expenses, and spending indicators
Plan Affordable Goals	Target amounts, deadlines, contributions, and conflict alternatives
Explore a Decision	Scenario comparisons and financial impact summaries
Retain Previous Choices	Decision records and relevant memory retrieval
Control Recommendations	Review actions for acceptance, modification, or rejection
Protect Personal Records	JWT authentication and user-scoped service queries

Nonfunctional requirements include reproducible arithmetic, readable explanations, consistent input validation, isolation between accounts, and manageable response latency. The present verification addresses software behavior; user comprehension and accessibility need a separate usability study.

4 Proposed Agentic AI Solution

4.1 Overall solution

FinPilot combines a web application, service layer, deterministic calculation engine, and coordinated agent workflow. Financial records are stored as structured data. Before an advisor request executes, a context builder assembles profile information, expense summaries, cash flow metrics, goals, and health indicators. This snapshot supplies the numerical basis for the specialized processing that follows.

The orchestrator classifies the request and selects an agent path. Spending requests use the spending analyst, allocation and goal requests use the planning agent, and affordability requests use the decision agent. Monitoring requests can lead to replanning and then planning when a drift assessment indicates that adaptation is required. Other supported paths include expense reduction and general advice. Output checks and reflection run before the response is returned.

4.2 Key features

Feature	Purpose
Financial Dashboard	Summarize health, available capacity, and key metrics
Transaction Intelligence	Parse entries, preview statement imports, and analyze spending
Salary & Goal Planning	Allocate income and evaluate funding feasibility for targets
Scenario Lab	Compare potential decisions against the financial baseline
AI Advisor	Explain verified context via the appropriate agent execution path
Decision Memory & Monitoring	Retain decision assumptions and detect drift affecting prior plans
Agent Lab & Human Review	Inspect execution traces and record structured user feedback

4.3 Agent based approach

The architecture separates the responsibility for selecting work from the responsibility for performing calculations and explaining them. Agents are software nodes with a defined input state, role, and returned updates. The graph coordinates these nodes rather than allowing unrestricted tool execution. The application registry contains eight roles including the orchestrator; validation, reflection, retrieval, and memory are supporting workflow components rather than additional financial specialists.

4.4 Scope of autonomy

The system supports analysis, proposed allocations, and decision assistance. Human review records a user’s response to a recommendation. An accepted review is a stored decision record and should not be interpreted as execution of a payment or a transfer. This distinction keeps the implemented behavior consistent with the project’s decision support purpose.

5 Dataset and Knowledge Sources

5.1 User financial data

The main operational data consists of user supplied financial profiles, transaction entries, salary settings, recurring commitments, budgets, and goals. Profiles include monthly income, current savings, essential expenses, debt payments, emergency savings, dependents, and risk preference. Transactions contain an amount, income or expense type, category, description, and date. Goals contain target and saved amounts, deadline, priority, contribution, and status. These records are inputs to calculations rather than a labeled training corpus.

Statement import supports CSV, Excel, and PDF processing. The implementation normalizes records, identifies likely column mappings, generates fingerprints, previews errors and duplicates, and accepts confirmation before import. Natural language transaction parsing provides extracted fields and missing field information. Voice entry relies on browser support where available; it is not a separately trained speech recognition model in the backend.

5.2 Local knowledge corpus

Repository Document	Category	Chunks
`rbi_financial_education.json`	Financial Literacy	3
`sebi_investor_guidelines.json`	Goal Planning	3
`indian_tax_and_budgeting.json`	Personal Budgeting	3
`debt_and_emergency_rules.json`	Debt and Reserves	3

The corpus contains four JSON documents and twelve prewritten chunks. Document metadata includes a source identifier, title, publisher, URL, category, type, and publication date. Chunks include text, titles, identifiers, and tags. The files are curated local material carrying publisher labels and links; the source code does not establish that every sentence is an exact quotation from the linked institution. Their attributed claims therefore require source verification before public deployment.

5.3 Evaluation cases and demonstration data

Five JSONL files contain thirteen authored evaluation questions: three spending, three decision, two planning, two monitoring, and three retrieval questions. Expected intent, tools, facts, and retrieval needs are included. This small development set cannot establish generalization. A suitable demonstration account can use ₹75,000 monthly income and ₹1,80,000 savings, with clearly labeled synthetic transactions and goals. Test fixtures must distinguish salary forecasts from received income.

6 Tools and Technologies Used

Technology / Library	Role in Implementation
React & TypeScript	Component-based UI and typed client data contracts
Vite & Tailwind CSS	Frontend build tooling, bundling, and utility-first styling
Recharts & Lucide	Financial data visualizations and UI iconography
FastAPI & Pydantic	REST API endpoints, dependency injection, and data validation
SQLAlchemy & Alembic	Relational ORM persistence and database schema migrations
PostgreSQL & SQLite	Production database target and local testing/development storage
LangGraph	Stateful agent orchestration and conditional graph transitions
Groq & LangChain Groq	High-speed LLM inference interface for explanation synthesis
Python Decimal	Exact financial currency calculations and explicit rounding
OpenPyXL & PyPDF	Excel spreadsheet and PDF financial statement parsing
JWT & bcrypt	Cryptographic token authentication and password hashing
Pytest & FastAPI TestClient	Automated integration testing for calculation engine and API routes

6.1 Technology selection rationale

The interface and API are separated so that views can request profile data, numerical summaries, and advisor outputs through defined endpoints. Pydantic schemas reject unsupported types and invalid amounts before business logic runs. SQLAlchemy provides a common model layer for the configured database. Alembic tracks schema evolution, including additions for decision memory, retrieval, reviews, and evaluation records.

LangGraph provides conditional transitions and a shared state structure for specialist processing. The language model interface is optional: the application falls back to deterministic response generation when live inference is unavailable or mock mode is selected. This supports repeatable tests, but a mock response must not be described as an observed output of the configured Groq model.

6.2 Verification environment

Local verification used Python 3.12, a fresh installation of the backend requirements, SQLite, and MOCK_LLM_MODE enabled. The frontend was installed from its package lock and built using the repository build command. PostgreSQL integration and live provider performance were not independently measured in this run. Backend requirements use lower version bounds, so recording the tested dependency environment is important for reproducibility.

7 Agent Architecture

7.1 System components

FinPilot AI consists of six connected components: the user interface, backend API, application services, financial engine, agent coordination layer, and persistence layer. Together, these components convert user inputs into financial calculations, planning recommendations, and explanatory responses. Figure 1 illustrates their relationships and the supporting knowledge retrieval and language model interfaces.

The **user interface**, developed using React and TypeScript, provides a common workspace for entering financial information and viewing results. Users can manage their profile, record transactions, define savings goals, explore scenarios, and ask financial questions. The interface communicates with the FastAPI backend through API requests. Authentication identifies the current user, while Pydantic schemas validate incoming data before application services process it.

The **application service layer** connects stored financial records with the calculation and agent components. For an advisor request, the financial context builder gathers profile information, transactions, budgets, and goals, then assembles the relevant financial indicators. This shared context gives the agents a consistent starting point for analysing the user's situation.

The **deterministic financial engine** performs calculations for cash flow, spending patterns, emergency coverage, goal contributions, and scenario comparisons. Its functions use Decimal arithmetic for monetary calculations. Separating these calculations from language generation makes the numerical methods independently testable and allows different agents to reuse the same financial tools.

The **agent coordination layer**, implemented using LangGraph, routes requests to specialist agents and manages updates to shared state. Depending on the request, the workflow may perform spending analysis, decision evaluation, planning, monitoring, or replanning. Validation and reflection stages inspect the resulting output, while execution traces record the processing steps. The language model interface provides explanations through Groq inference or deterministic mock responses.

The **persistence and knowledge components** support continuity across requests. SQLAlchemy manages financial records, decision memory, knowledge chunks, review records, and audit events. PostgreSQL is the configured deployment database, while SQLite supports local testing. The retrieval component searches the curated financial knowledge corpus and attaches relevant chunks and citations to agent state. In the reviewed implementation, passage injection into the language model prompt remains incomplete, so retrieved citations should not automatically be interpreted as evidence that the generated explanation used those passages.

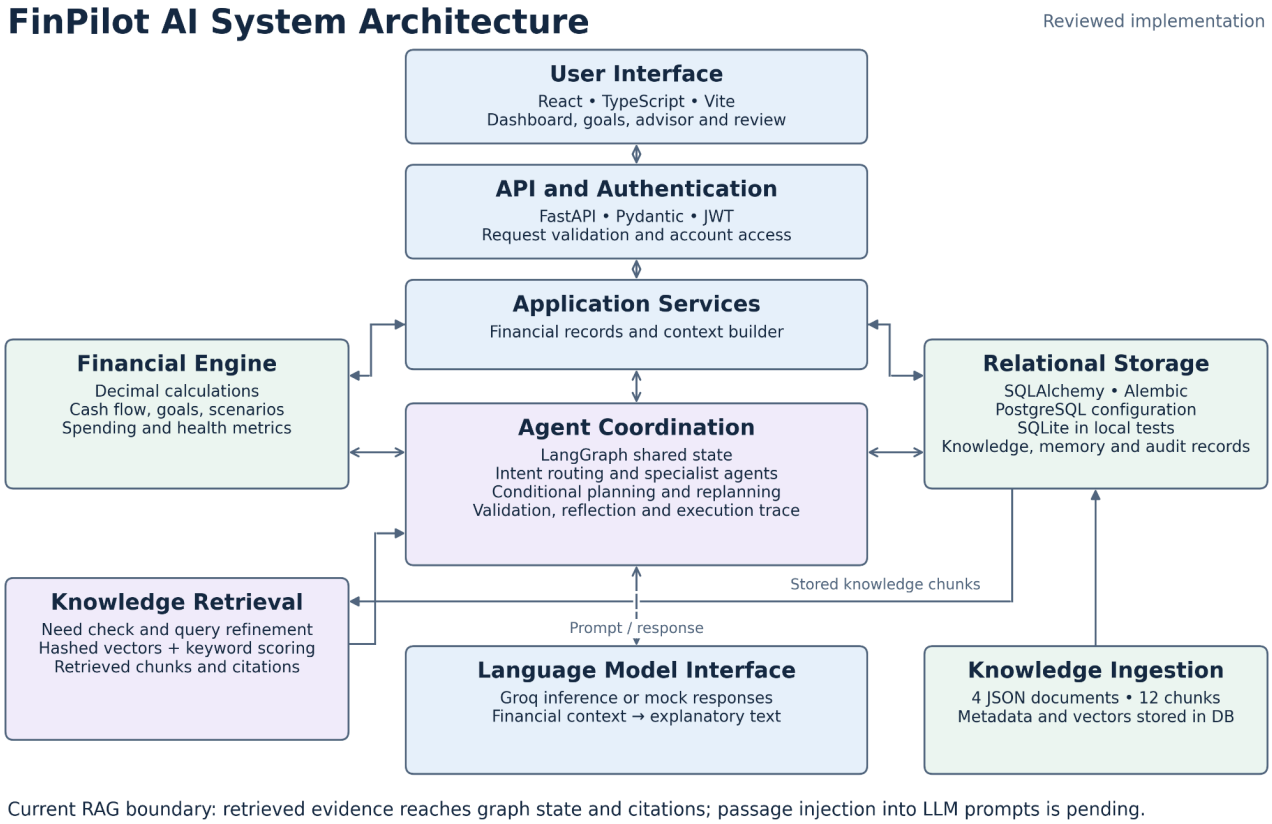


Figure 1 System architecture showing frontend services tools graph and storage

7.2 Shared state and context

AgentState includes the request and user identifiers, query, financial context, memory context, intent, selected agents, chain, facts, recommendations, scenario and planning results, monitoring and replanning fields, retrieval citations, validation errors, reflection audit, and execution trace. Nodes return state updates. This gives downstream nodes access to preceding calculations without requiring them to reconstruct the entire user profile.

7.3 Tool boundary

The numerical engine acts as a reusable tool layer. Agents call or interpret scenario, allocation, spending, and monitoring functions. This division makes arithmetic independently testable. The design reduces dependence on model generated numbers, but correctness still depends on validated inputs, suitable rules, and complete output checks. Decimal arithmetic alone does not guarantee that every recommendation is correct.

7.4 Agent roles and responsibilities

Agent Role	Inputs & Core Responsibility	Principal Output
Orchestrator	User query and shared context; classifies user intent and routes execution	Detected intent, agent chain, and routing trace
Spending Analyst	Transaction history and categories; identifies outflow drivers and patterns	Spending facts and category-level insights
Expense Reduction	Discretionary and recurring spending; evaluates cost-cutting targets	Potential monthly savings and action steps
Decision Intelligence	Decision parameters and baseline; projects financial trade-offs	Scenario evaluations and trade-off metrics
Planning Agent	Income, obligations, and goals; assesses resource allocation feasibility	Allocation strategy and goal funding path
Monitoring Agent	Current state vs baseline plans; measures financial drift	Drift score and replanning triggers
Adaptive Replanning	Detected state variances and affected targets; builds recovery paths	Comparative recovery options
General Advisor	Complete financial context; handles general queries and guidance	Contextual explanatory synthesis

7.5 Prompts and reasoning

Specialist prompt modules define role instructions and version identifiers. The shared inference helper supplies verified financial context and instructs the model not to invent balances or rates. The workflow exposes operational steps such as context construction, agent selection, scenario calculation, and validation. These execution events explain system behavior without requiring private internal model reasoning to be displayed.

7.6 Implementation boundaries

The orchestrator uses ordered keyword matching rather than a learned intent classifier. Earlier rules can take precedence when a query matches several patterns. Targeted verification selected planning_agent with the query “Analyze my spending” but returned spending_analyst as the actual selected agent. A nonexistent agent identifier was also accepted. Agent Lab run endpoints return the requested identifier as metadata while executing the common query routed graph; explicit selection is therefore not implemented reliably.

7.7 Cash flow and reserve calculations

The deterministic engine calculates disposable income by subtracting essential expenses, debt payments, and any supplied discretionary expenses from monthly income. Monetary results are quantized to two

decimal places. Savings rate is the surplus divided by income, expressed as a percentage. Debt to income is the monthly debt payment divided by income. Zero income is handled explicitly to avoid invalid division.

Disposable income = monthly income − essentials − debt payments − other discretionary expenses.
Savings rate = 100 × monthly savings or surplus ÷ monthly income.
Emergency coverage = reserve amount ÷ monthly essential expenses.

For an illustrative input of ₹75,000 income, ₹30,000 essentials, ₹5,000 debt payment, and ₹10,000 discretionary expense, disposable income is ₹30,000 and the savings rate is 40%. A reserve of ₹1,20,000 against ₹30,000 monthly essentials provides four months of coverage. These examples validate the intended arithmetic and are separate from the service integration defects found in testing.

7.8 Goals and safe spending

The goal engine calculates required contributions from remaining targets and available months, and proposes alternatives when goals compete for capacity. The safe spending engine supports deduction of commitments, essential allowances, savings reserves, and emergency reserves. However, salary_service currently adds recorded income to configured salary and does not pass the profile's emergency reserve into this calculation. The formula's capability must therefore be distinguished from the API's current behavior. These defects must be corrected before treating displayed spendable funds as reliable.

7.9 Financial health score

The implemented default score uses five components: savings rate at weight 0.25, debt to income at 0.25, emergency fund at 0.20, budget discipline at 0.15, and goal progress at 0.15. The result is bounded between zero and one hundred. Savings receives its maximum component score at a 20% rate; debt receives its maximum at or below 20% debt to income and reaches zero at 50%. Emergency coverage reaches its maximum at six months.

When budget discipline and goal progress are not supplied, the function uses default scores of 85 and 80 respectively. The health score is a project heuristic, not a validated credit score or externally certified financial health measure. These defaults and thresholds should be stated when interpreting the output.

8 Agent Workflow and Orchestration

8.1 Workflow design

The advisor entry function builds deterministic context, executes the retrieval need policy, and retrieves up to five relevant decision records. It creates initial graph state, after which the graph records orchestration, attaches prepared retrieval and memory, dispatches a specialist, validates structured outputs, and runs reflection. Figure 2 is reserved for the conditional graph, including spending to planning and monitoring to replanning to planning.

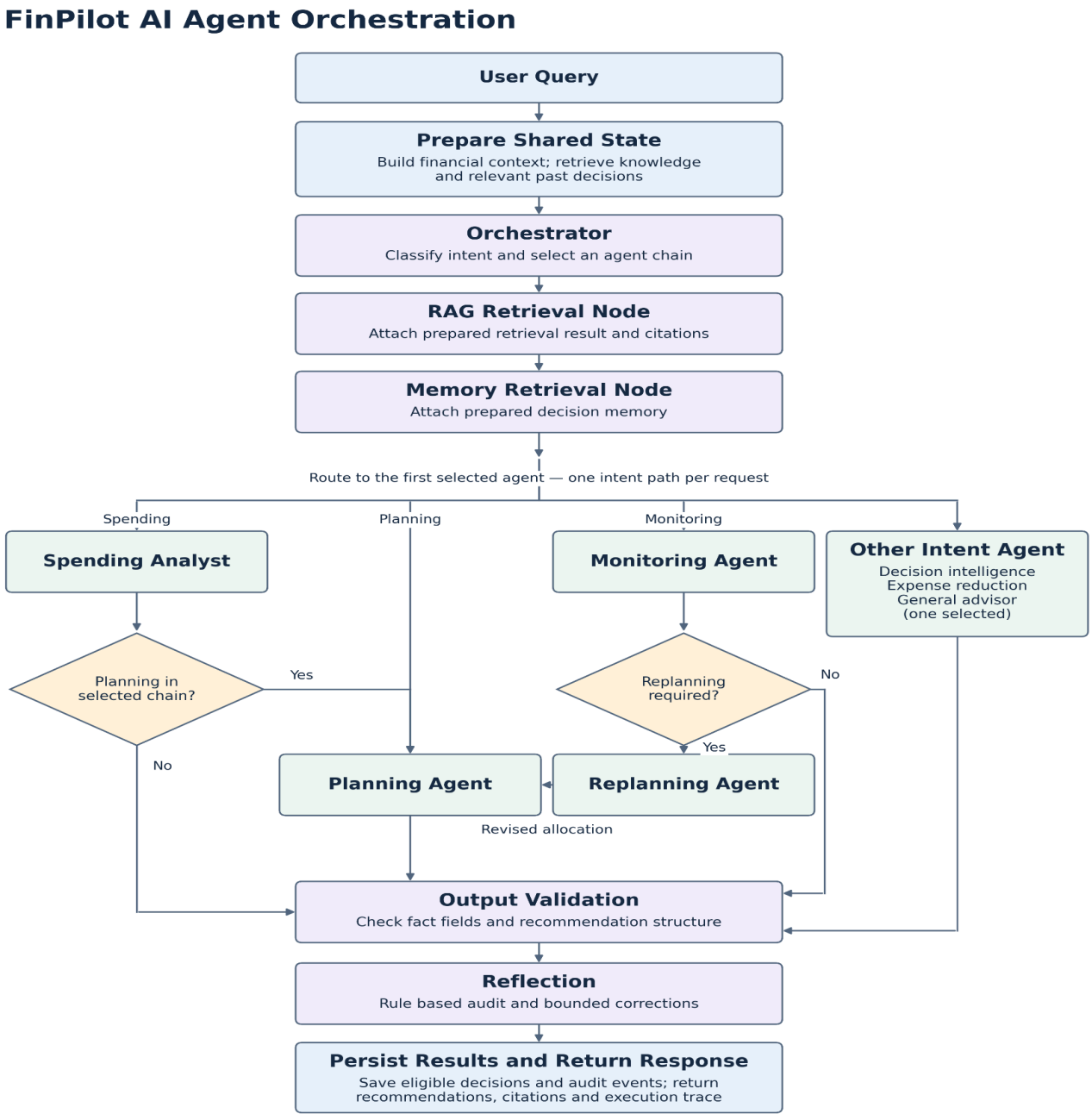


Figure 2 Conditional agent workflow with validation and reflection

8.2 Task decomposition and coordination

A spending question that also mentions goal impact can chain the spending analyst into planning. A monitoring request first calculates an assessment. If adaptation is required, processing passes through replanning and then planning; otherwise it proceeds to validation. Decision, expense reduction, and general advisor paths normally proceed directly to validation after their specialist output. The graph is a bounded conditional workflow, rather than an unrestricted loop of agents exchanging messages.

The final workflow diagram should show the bypass from monitoring to validation when adaptation is unnecessary. Decision, expense reduction, and general advice should have their own specialist branches. The returned trace records the path actually executed for each request and should be compared against the requested task during testing.

8.3 Persistent memory

DecisionHistory stores structured information about a prior decision, including its type, item, amount, chosen strategy, alternatives, affected goals, baseline metrics, resulting metrics, assumptions, and recommendation summary. Relevant records are retrieved before the advisor graph runs and attached to state. This provides continuity across requests and makes prior commitments available for reconsideration.

The entry function automatically attempts to save decisions for selected intents, including decision evaluation. Stored fields are derived from available state and include some default labels. A saved record therefore documents the system’s recommendation and context; it is not necessarily proof that the user performed the recommended action. Persistence failures are logged separately, so a returned response does not guarantee that all supplementary records were saved.

8.4 Monitoring and adaptation

The monitoring layer compares the current financial state with plan and decision assumptions. Its assessment identifies changes and determines whether replanning is required. The replanning node develops recovery alternatives and passes control to the planning node for revised allocation. This allows a single request to connect diagnosis and adaptation while preserving a trace of the transition.

A typical demonstration changes a profile or transaction record, requests a plan review, and inspects the before and after comparison. The prototype’s monitoring behavior is request driven. A future scheduled service could make these checks periodic, but scheduling, notifications, and sustained unattended operation require additional implementation and evaluation.

8.5 Human in the loop

Recommendations receive a pending review status during reflection. Authenticated review endpoints accept the actions accepted, modified, or rejected and store notes, the original recommendation, and any modified form. Review history is scoped to the current user. This records human feedback and creates an inspectable trail of decisions.

The endpoint records feedback but does not apply a budget change or execute a transaction. A targeted API probe accepted recommendation_id “does-not-exist” and returned success, showing that existence and ownership of the referenced recommendation are not checked. A stronger implementation should persist recommendations, link reviews to owned records, and validate modifications before application.

9 Agentic RAG Implementation

9.1 Ingestion and storage

The ingestion pipeline reads JSON knowledge files and upserts document metadata and chunks by source identifier and chunk index. The repository already supplies chunk boundaries, so the implementation does not demonstrate automatic segmentation of arbitrary external documents. Each chunk receives an embedding stored as a serialized JSON vector alongside text and tags. Relational storage holds the document catalog, chunks, and retrieval events.

9.2 Text representation and hybrid scoring

The embedding service constructs 384 dimensional vectors through feature hashing of token unigrams and bigrams. SHA 256 determines each feature’s position and sign; logarithmic frequency weighting is applied and the vector is normalized. This representation is deterministic and inexpensive for a small local corpus. It is a lexical representation, not a pretrained semantic embedding model.

The retriever scores candidate chunks using the normalized vector dot product, clamped to the range zero to one, and adds a keyword and tag overlap bonus capped at 0.30. Final scores are capped at one, filtered by a threshold, sorted, and truncated to the requested top k. The default top k is three. Category filtering reduces the candidate set when a relevant category is selected.

9 3 Agentic retrieval policy

9.4 Citations and logging

Sufficiency is a heuristic based on the top retrieval score divided by 0.50 and capped at one. It is not a calibrated probability of correctness. This policy supports one bounded refinement, which makes its execution easy to trace. It does not demonstrate an open ended model controlled search loop.

9.3 Agentic retrieval policy

Citations are deduplicated by source identifier and include title, publisher, category, URL, and a short relevant excerpt. Retrieval events store the question, selected category, chunk identifiers, sufficiency, refinement count, sources, and latency. These records support diagnosis of the retrieval stage and inspection of what evidence was available.

9.5 Generation integration

The advisor entry function places retrieved chunks and citations into shared state. The graph's retrieval node exposes the prepared result and records a trace step. Citations are passed to the reflection audit and returned in the advisor output. However, the reviewed specialist calls to the shared inference helper supply financial_context; that helper constructs its prompt from the supplied context and does not directly read the RAG state fields.

A component probe supplied a unique retrieved passage in rag_chunks and captured the arguments to GeneralAdvisorAgent's inference helper. The passage and citations were absent from the supplied generation context. The current implementation therefore provides retrieval and citation attachment, while passage grounded generation is incomplete. Citation presence alone does not demonstrate that a generated claim used or was supported by the retrieved evidence.

9.6 Existing evaluation models

Evaluation Mode	Implemented Retrieval Behavior
No RAG	Bypasses retrieval entirely and returns empty context results
Basic RAG	Searches all queries uniformly without category filtering at threshold 0.10
Agentic RAG	Evaluates retrieval need, reformulates query, filters by category, and executes conditional retry if score is low

The evaluator runs the retrieval component and checks whether chunks were returned under mode specific conditions. It constructs simulated explanatory content for a deterministic reflection audit. Intent and tool matches are marked successful without comparing actual execution against the expected labels. Structured output validity is fixed at one, and latency contains an added mode dependent constant. These outputs are useful for demonstrating an evaluation interface but are not independent end to end measures of model performance.

9.7 Appropriate quality metrics

A rigorous retrieval experiment should annotate which chunks are relevant to each question. Precision at k would measure the fraction of returned chunks that are relevant; recall would measure the proportion of relevant evidence recovered. Retrieval need accuracy should compare the policy decision with an expected label. Citation correctness should assess whether the cited passage supports a claim, and answer groundedness should be evaluated on the actual generated answer.

The thirteen development questions provide a starting point, but broader held out cases, paraphrases, ambiguous requests, unsupported questions, and conflicting documents are needed. No measured superiority of agentic retrieval over basic retrieval is claimed in this report.

10 System Implementation

10.1 Functional prototype

The frontend contains Home, My Money, My Plan, AI Advisor, Agent Lab, Scenario Lab, Insights, and Settings. Authentication controls access to user views. The following spaces are reserved for screenshots of the corrected running prototype. Capture the dashboard and planning screens using a synthetic account and label the provider mode used.

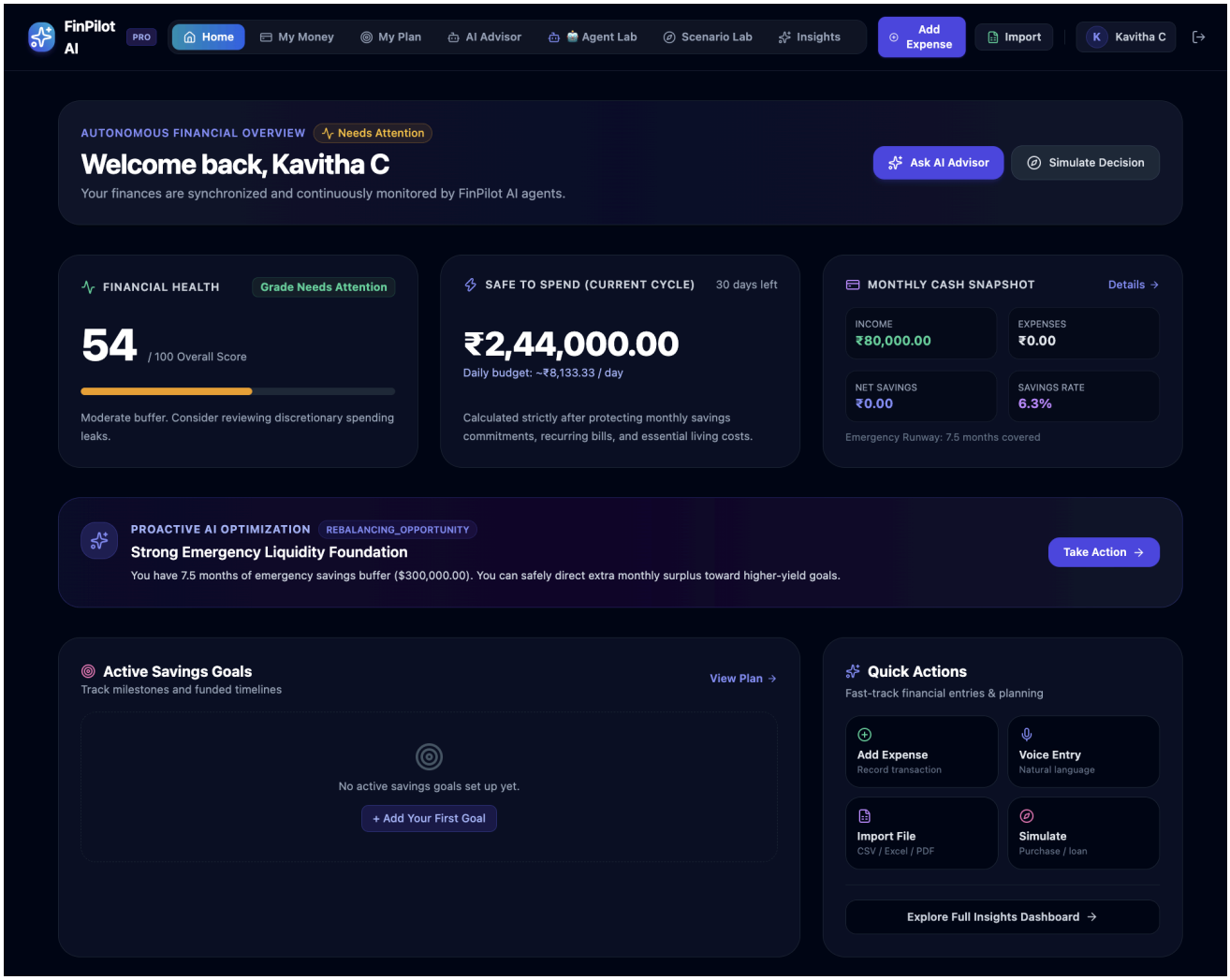


Figure 3 Financial dashboard with a synthetic account

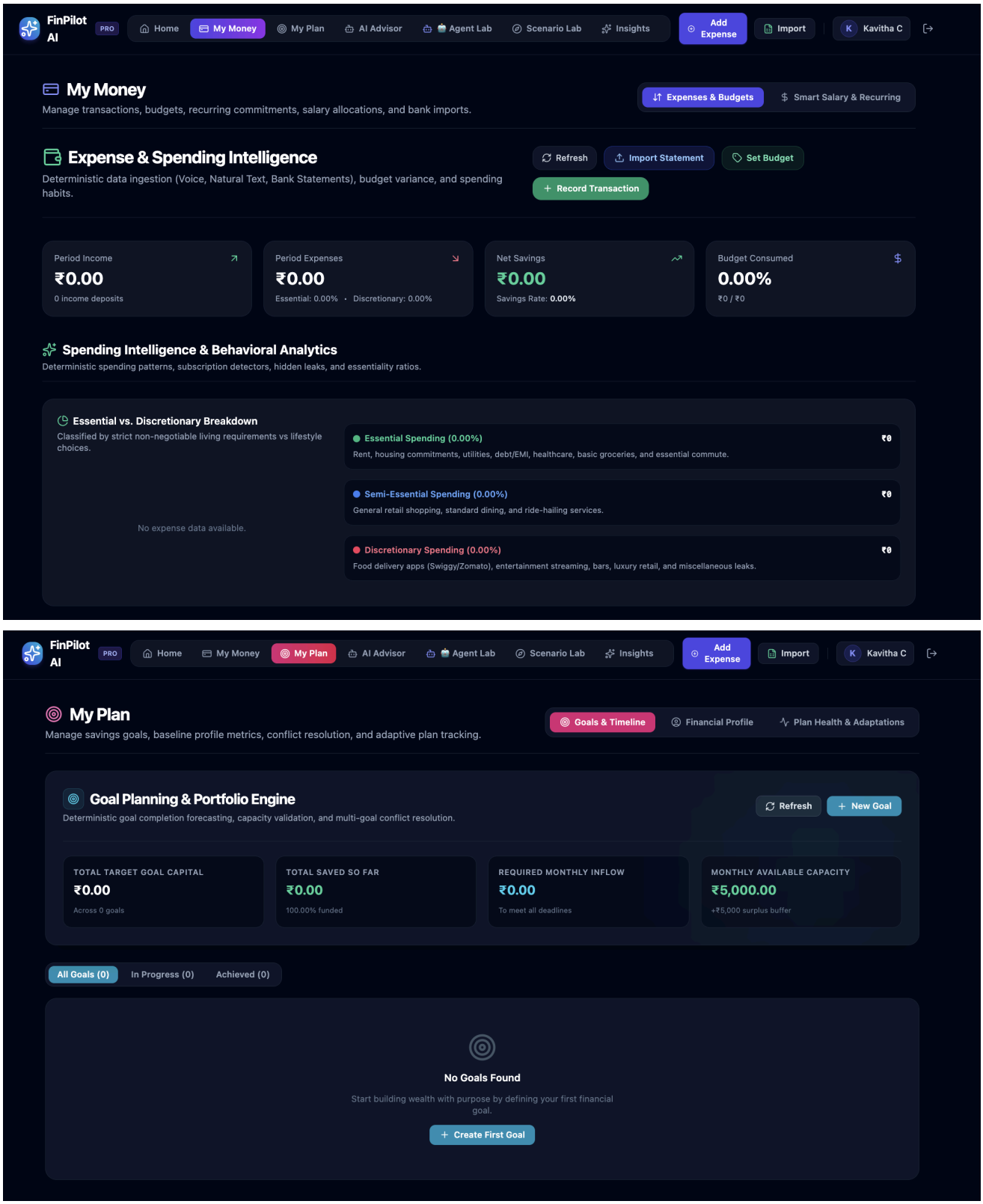


Figure 4 Goals and allocation view

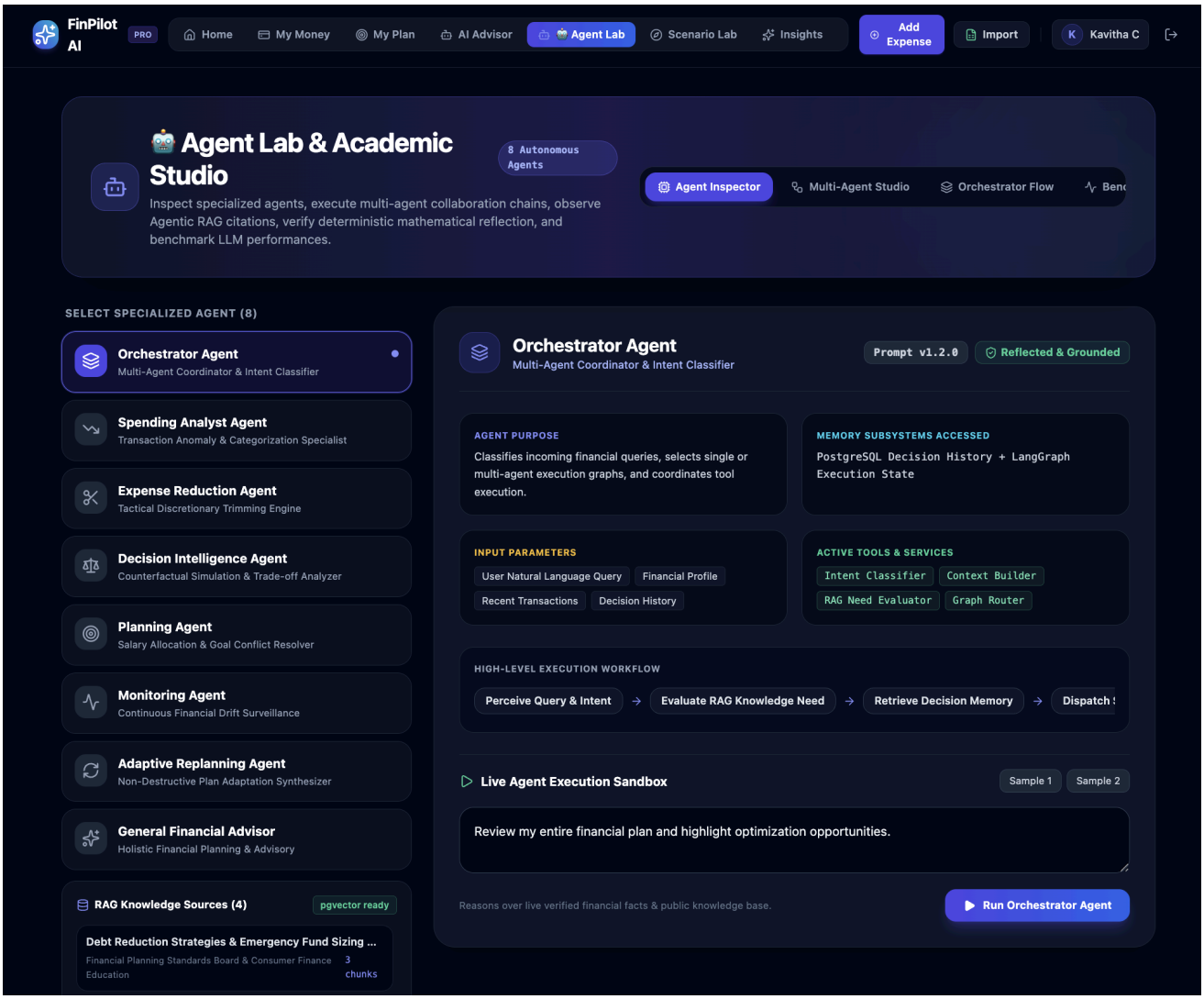


Figure 5 Agent Lab with requested agent and actual execution trace

10.2 Backend modules and API boundaries

Module / Route Group	Primary Responsibility
auth & profile	User account authentication and financial profile baseline maintenance
transactions & expenses	Transaction entry, statement imports, categorization, and spending summaries
salary, budgets & goals	Cash flow parameters, budget allocations, spending limits, and savings targets
scenarios & decisions	What-if scenario modeling and decision history record persistence
advisor & monitoring	LangGraph workflow execution, advice synthesis, and drift assessments
rag, hitl & evaluation	Knowledge base retrieval, human review tracking, and evaluation benchmark APIs

FinPilot AI is implemented using a React and TypeScript frontend connected to a FastAPI backend. The backend separates API routes, input schemas, application services, financial calculations, agent coordination, knowledge retrieval, and database models. This structure allows financial calculations and agent workflows to be developed and tested independently. SQLAlchemy manages database operations, while Alembic maintains schema migrations.

10.3 Source Code Repository and Execution

The source code for FinPilot AI is maintained in a GitHub repository. The repository contains the frontend application, backend services, financial engine, agent workflow, knowledge datasets, evaluation cases, and automated tests. The frontend is started using Vite, while the backend runs through Uvicorn. Database settings and language model configuration are supplied through environment variables.

Repository: https://github.com/marcellus878/FinPilot-AI

11 Experimental Evaluation and Results

11.1 Evaluation Objectives

The evaluation of FinPilot AI examines whether the system converts user financial information into consistent calculations, appropriate agent workflows, and understandable recommendations. It covers financial calculations, agent selection, workflow coordination, knowledge retrieval, decision memory, and human review.

The evaluation also considers whether the system responds appropriately when financial conditions change. This includes identifying changes affecting an existing plan, generating revised planning options, and presenting their implications to the user.

Targeted Probe / Check	Observed Result	Analytical Interpretation
Backend Test Suite & Build	169 tests passed in 4.27s; frontend build succeeded	Existing regression tests pass under local SQLite and mock LLM settings
Salary Accounting	₹2,55,000 before salary entry; ₹3,30,000 after entry	Double counting defect: configured base salary and logged salary are summed
Emergency Reserve	₹1,20,000 configured; ₹0 reported in spendable call	Integration gap: configured protected reserve is omitted from safe-spending calculation
Fact Validator	Accepted income 999999 against baseline context of 75000	Numerical consistency checks are not strictly enforced during reflection
Agent Lab Routing	Planning Agent selected; Spending Analyst executed	Explicit agent ID does not override natural language intent routing; invalid IDs accepted
RAG Generation Grounding	Retrieved passage absent from captured generation prompt context	General Advisor agent fails to inject retrieved RAG passages into generation context
Recommendation Review	Nonexistent recommendation ID accepted successfully	Lack of database entity existence and ownership validation on review endpoints

11.2 Testing Methodology

The evaluation combines automated testing with functional testing of the application. Automated tests assess financial calculations, input validation, API responses, account isolation, and agent transitions. Functional testing follows complete user journeys through profile creation, transaction entry, goal planning, scenario analysis, advisory interaction, and recommendation review.

Synthetic financial profiles provide controlled inputs with known expected results. Test scenarios include different income levels, recurring commitments, emergency reserves, competing goals, additional income, and unexpected expenses. Calculated outputs are compared against independently derived expected values.

Agent workflows are evaluated by comparing the requested task with the selected agents and returned execution trace. Retrieval testing uses financial questions with identified relevant knowledge passages. Generated responses are inspected for numerical consistency, relevance, and support from the cited evidence.

11.3 Evaluation Metrics

Metric	Purpose
Financial calculation correctness	Checks whether calculated amounts match expected results
Intent-routing accuracy	Measures whether queries reach the appropriate agent
Workflow completion rate	Measures whether expected agent sequences finish successfully
Retrieval precision at k	Measures the proportion of retrieved passages relevant to the question
Citation support rate	Checks whether cited passages support the associated claims
Structured-output validity	Measures whether responses follow the required output format
Response latency	Measures the time taken to complete a request
Functional scenario success rate	Measures whether complete user journeys produce the expected outcome

11.4 Benchmark Limitations

The model comparator returns constant accuracy and latency values. The retrieval evaluator includes assumed successes, simulated content, and artificial timing offsets. These are demonstration outputs, excluded from measured results. A displayed 100% value is not independently measured model accuracy.

The fact validator accepted Monthly Income 999999 against context 75000. Collected reference values are not compared. Reflection makes limited rule based corrections; claim level numerical verification remains unfinished.

12 Comparison Innovation and Scalability

Dimension	Traditional Dashboard / Generic Chat	FinPilot Agentic Solution
Numerical Computation	Static database totals or LLM-generated approximate arithmetic	Dedicated, deterministic Python Decimal financial calculation tools
Task Execution	Fixed UI views or single-turn conversational text generation	Multi-agent conditional specialist graph with validation & reflection
Decision Continuity	Isolated session context or basic transaction history log	Persistent structured decision memory and baseline context
Adaptive Replanning	Manual plan adjustments required by the user upon drift	Automated drift monitoring triggers adaptive replanning nodes
Process Transparency	Opaque "black box" model responses with limited visibility	Inspectable graph execution traces, citations, and human review logs

This is a design comparison, not a measured comparative trial. The main engineering contribution is integration of numerical tools, conditional coordination, memory, and user review in one prototype. Graph state and API traces make the processing path inspectable. A stronger baseline experiment would execute the same held out questions through a single agent and the multi agent graph under equal model and tool conditions.

The separated client and API can support additional deployments, but scalability has not been load tested. Retrieval currently loads candidate chunks into application memory and scores them in Python. Larger corpora would benefit from indexed search and versioned ingestion. Background jobs, caching, pagination, request limits, and frontend code splitting would improve operation with more users and records. Knowledge provenance and user scoped data handling remain essential as the system grows.

13 Conclusion and Future Work

FinPilot AI demonstrates a connected financial planning prototype with numerical tools, specialist coordination, scenario analysis, decision memory, request driven monitoring, local retrieval, and recorded human feedback. Its 169 passing tests and successful build confirm execution under the documented conditions. Targeted checks expose important integration defects, so the current evidence supports a working prototype with unfinished correctness and grounding requirements.

Immediate work should correct salary accounting and emergency reserve propagation, enforce numerical fact validation, route explicit Agent Lab selections, and supply retrieved passages to generation. Review linkage and genuine evaluation must also be completed. Longer term work includes verified knowledge provenance, scheduled monitoring, broader held out experiments, usability studies, load testing, indexed retrieval, and validation of financial health thresholds.

References

[1] Marcellus878. FinPilot AI source repository. Reviewed revision 80332ca68f0fd4b05966dd6dea591fcf2dd8110e, 2 October 2026. https://github.com/marcellus878/FinPilot-AI

[2] LangChain. LangGraph documentation. Reference for shared state, nodes, and conditional workflow orchestration. https://docs.langchain.com/oss/python/langgraph/overview

[3] FastAPI. Official documentation. Reference for typed APIs, validation, and dependency injection. https://fastapi.tiangolo.com/

[4] Python Software Foundation. Decimal module documentation. Reference for decimal arithmetic and rounding. https://docs.python.org/3/library/decimal.html

[5] SQLAlchemy. SQLAlchemy ORM documentation. Reference for relational models, sessions, and queries. https://docs.sqlalchemy.org/en/20/orm/

[6] Groq. Groq API documentation. Reference for the configured language model provider. https://console.groq.com/docs/overview

[7] Lewis, P., et al. Retrieval Augmented Generation for Knowledge Intensive NLP Tasks. Advances in Neural Information Processing Systems, 2020. https://arxiv.org/abs/2005.11401

Repository evidence used

Primary evidence includes backend/app/services/salary_service.py; backend/app/agent/graph.py, orchestrator.py, general_advisor.py, llm.py, validator.py, reflection.py; financial_engine/core_metrics.py, health_score.py, constants.py; rag/ingestion.py, embeddings.py, retriever.py, agentic_retrieval.py; evaluation/evaluator.py, llm_comparator.py; api/routes/agent_lab.py, hitl.py; frontend/src/App.tsx; datasets; and backend/tests.

Knowledge source provenance

Repository knowledge metadata links to RBI financial education, SEBI investor education, NIFM, and FPSB websites. These links identify the repository’s stated source associations. They are not used here to certify that every locally curated sentence is an official institutional recommendation. Exact document references, passage checks, licensing confirmation, and version dates should be added when the corpus is verified.